# Edge Intelligence

# Lab Assessment

# Feb 19 2026

**M Rahul – 25MML0011**

## Code 1 :-

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# 1. Load the pretrained MobileNetV2 model once
# MobileNetV2 is optimized for speed on mobile and edge devices.
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    """
    Opens a video file and extracts a specific number of frames.
    """
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print(f"Error: Cannot open video file {video_path}")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Calculate step size to sample frames evenly across the video
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
```

```python
        if count % step == 0:
            # Resize to 224x224 as required by MobileNetV2
            frame = cv2.resize(frame, (224, 224))
            # OpenCV reads in BGR; convert to RGB for proper classification
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

        count += 1
        if len(frames) >= max_frames:
            break

    cap.release()
    print(f"Extracted {len(frames)} frames.")
    return np.array(frames)

def classify_video(video_path):
    """
    Extracts frames, runs them through the model, and averages the results.
    """
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames available for classification.")
        return

    # 2. Preprocess the frame data for the model
    # This scales pixel values according to MobileNetV2 requirements.
    processed_frames = preprocess_input(frames.astype('float32'))

    # 3. Predict labels for all extracted frames
    predictions = model.predict(processed_frames)

    # 4. Average predictions across all frames to get a stable video label
    avg_pred = np.mean(predictions, axis=0)

    # 5. Decode results into human-readable ImageNet labels
    top_preds = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    print("\n--- Top Predictions ---")
    for i, (imagenet_id, label, score) in enumerate(top_preds[0]):
        print(f"{i+1}. {label}: {score*100:.2f}%")

    print(f"\nFinal Video Prediction: {top_preds[0][0][1]}")
```

```python
# --- Execution ---
# Replace with the actual path to your video file
video_path = "/content/sample_video.mp4"

# Tip: If in Google Colab, upload the video or use !wget to download one first.
classify_video(video_path)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5
14536120/14536120 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Extracted 20 frames.
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

--- Top Predictions ---
1. digital_clock: 30.81%
2. screen: 6.22%
3. monitor: 5.01%
4. web_site: 4.31%
5. analog_clock: 3.65%

Final Video Prediction: digital_clock
```

```
Extracted 20 frames.
Frames array shape: (20, 224, 224, 3)
Raw Pixel [0,0,0] (RGB): [0 0 0]
Processed Pixel [0,0,0]: [-1. -1. -1.]
```

**Steps we Implemented :-**

- Loaded pretrained **MobileNetV2** model.
- Extracted 20 evenly spaced frames from the video.
- Resized frames to 224×224 and converted BGR to RGB.
- Preprocessed frames using preprocess_input().
- Predicted 1000-class probabilities for each frame.
- Averaged all frame predictions to get one video-level output.
- Decoded and printed top 5 ImageNet labels.

**Drawbacks :-**

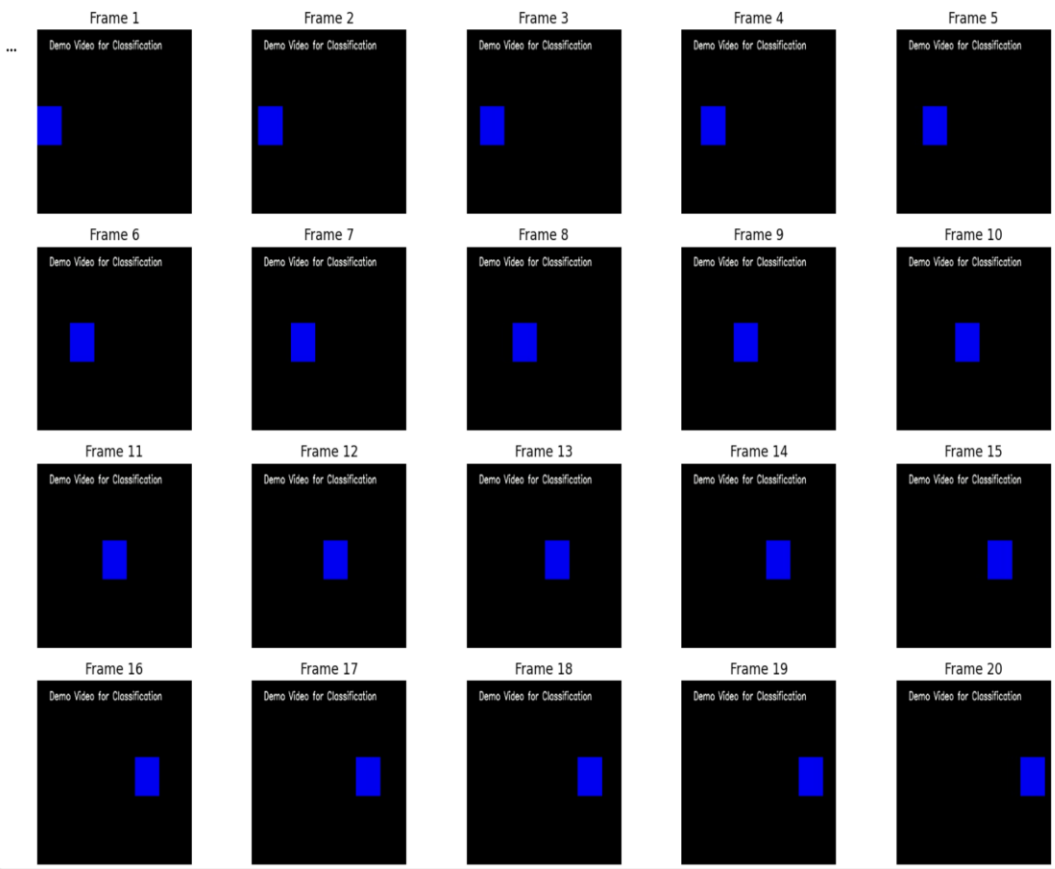
- **MobileNetV2** is an image model, not a video model.
- It is trained only on **ImageNet** (1000 object classes).
- The actual video class may not exist in ImageNet categories.
- CNN detects visual patterns, not semantic meaning.
- Averaging frames can amplify irrelevant objects.
- The model confidence (30%) shows uncertainty, not strong prediction.

**Errors in the code :-**

- Duplicate imports of cv2 and numpy (redundant but not breaking).
- tensorflow as tf is imported but never used.
- No explicit frames.astype('float32') before preprocess_input() (may cause dtype inconsistency).
- No check if video file path actually exists before opening.
- model.predict() does not set verbose=0, so it prints progress unnecessarily.
- Frame sampling using total_frames // max_frames may skip important frames.
- No GPU memory handling (can crash for large frame counts).
- No exception handling around model loading or prediction.
- Averaging predictions ignores temporal information (logical limitation).
- Model is trained for images, but used for video classification (design issue, not syntax error).

```python
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
import sys

# 1. Load Model
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        return np.array([])
    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames)
    count = 0
    while True:
        ret, frame = cap.read()
        if not ret: break
        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)
        count += 1
        if len(frames) >= max_frames: break
    cap.release()
    return np.array(frames)

def classify_video(video_path):
    frames = extract_frames(video_path)
    if len(frames) == 0: return

    # --- PRINTING ALL 20 FRAMES ---
    print(f"Total Array Shape: {frames.shape}")

    # UNCOMMENT the line below ONLY if you want to see millions of numbers (NOT RECOMMENDED)
    # np.set_printoptions(threshold=sys.maxsize)

    for i in range(len(frames)):
        print(f"\n--- FRAME {i+1} PIXELS (Raw RGB) ---")
        # Printing the first 3x3 grid of pixels for each frame to keep it readable
        print(frames[i, 0:3, 0:3, :])

    # 2. Preprocess
    processed_frames = preprocess_input(frames.astype('float32'))

    print("\n--- PROCESSED FRAME 1 (First 3 pixels) ---")
    print(processed_frames[0, 0, 0:3, :])

    # 3. Predict
    predictions = model.predict(processed_frames)
    avg_pred = np.mean(predictions, axis=0)
    top_preds = decode_predictions(np.expand_dims(avg_pred, axis=0), top=3)

    print("\n--- Final Results ---")
    for i, (id, label, score) in enumerate(top_preds[0]):
        print(f"{label}: {score*100:.2f}%")

# --- Run ---
video_path = "/content/sample_video.mp4"
classify_video(video_path)
```

```
Total Array Shape: (20, 224, 224, 3)

--- FRAME 1 PIXELS (Raw RGB) ---
[[[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]]

--- FRAME 2 PIXELS (Raw RGB) ---
[[[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]]]
```




**Code 2 :-**

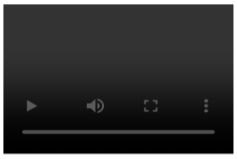
**Steps :-**

1. Import required libraries (cv2, numpy, matplotlib, subprocess, etc.).
2. Install OpenCV using pip install opencv-python (if not installed).
3. Import subprocess (optional, used for running system commands).
4. Provide the video file path.
5. Load the video using cv2.VideoCapture().
6. Check if the video opened successfully.
7. Read video metadata using cap.get() function.
8. Count total number of frames using cv2.CAP_PROP_FRAME_COUNT.
9. Get video frame width using cv2.CAP_PROP_FRAME_WIDTH.
10. Get video frame height using cv2.CAP_PROP_FRAME_HEIGHT.
11. Get frames per second (FPS) using cv2.CAP_PROP_FPS.
12. Read the first frame using cap.read().
13. Understand that OpenCV loads images in **BGR** format.
14. Convert BGR image to **RGB** using cv2.cvtColor() for matplotlib display.
15. Use modulo operator (frame_number % 100 == 0) to extract every 100th frame.
16. Display frames using matplotlib.pyplot.imshow().
17. Release video capture using cap.release().

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm
```

```
#import subprocess
```

```
ipd.Video("026c7465-309f6d33.mp4",width = 200)
```



```
Start coding or generate with AI.
```

Open the Video and Read Metadata

- List item
- List item

```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

```python
# Load in video capture
import cv2
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
```

```python
# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

```
2398.0
```

```python
# Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

```
Height 720.0, Width 1280.0
```

```python
# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')
```

```
FPS : 59.94
```

```python
cap.release()
```

Pull the image from the first frame

```python
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read()                          #will return TRUE everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

```
Returned True and img of shape (720, 1280, 3)
```

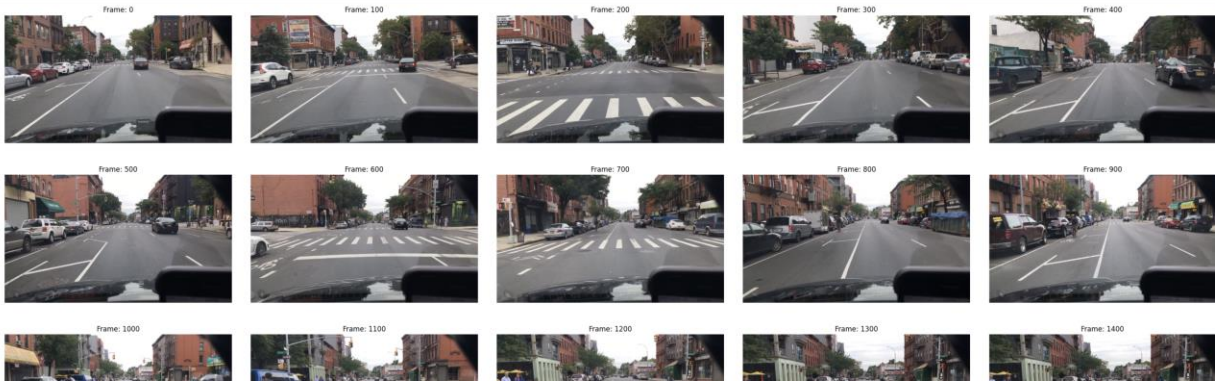
Loading the First Image

```python
plt.imshow(img)  #opencv loads color in BGR format, matplot lib expects RGB format
```

```
<matplotlib.image.AxesImage at 0x7faad2e85be0>
```

```python
## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")
```

```python
display_cv2_img(img)
```

```python
cap.release()
```

```python
import cv2
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:                                    #frame is divisible by 100 - Note the modulo operator
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'Frame: {frame}')
        axs[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()                                          #automatically adjusts the spacing between the plots
plt.show()
cap.release()
```